

PROJECT DOCUMENTATION

AI BLOGNEST API

INTRODUCTION

Project Title: **AI BLOGNEST API**

ID:NM2025TMID38547

TeamLeader:

Name: IshaMalini M&MailID:ishamalini77@gmail.com[Coder]

TeamMembers:

Name:GeethanjaliR&MailID:geethanjali065@gmail.com[Documentation]

Name:HanshiniBalaTD&MailID:hanshinibala@gmail.com[Coder]

Name:Harshini S&MailID:harshinisibi2811@gmail.com[Voiceover]

Name: Janani M &MailID: jananimunikrishnan@gmail.com

[Documentation]

Description

- AI BlogNest is an AI-powered blog management system developed to help users create and manage blog content easily. The project uses Node.js and Express.js for backend development and MongoDB for storing user and blog information.
- The system integrates Google Gemini AI to generate blog content based on the topic entered by the user. It also provides a blog summarization feature that helps users understand lengthy blog content in a shorter and simpler form.
- The backend supports user registration, login, blog creation, and blog retrieval. JWT is used for user authentication, while bcrypt.js is used for secure password hashing.

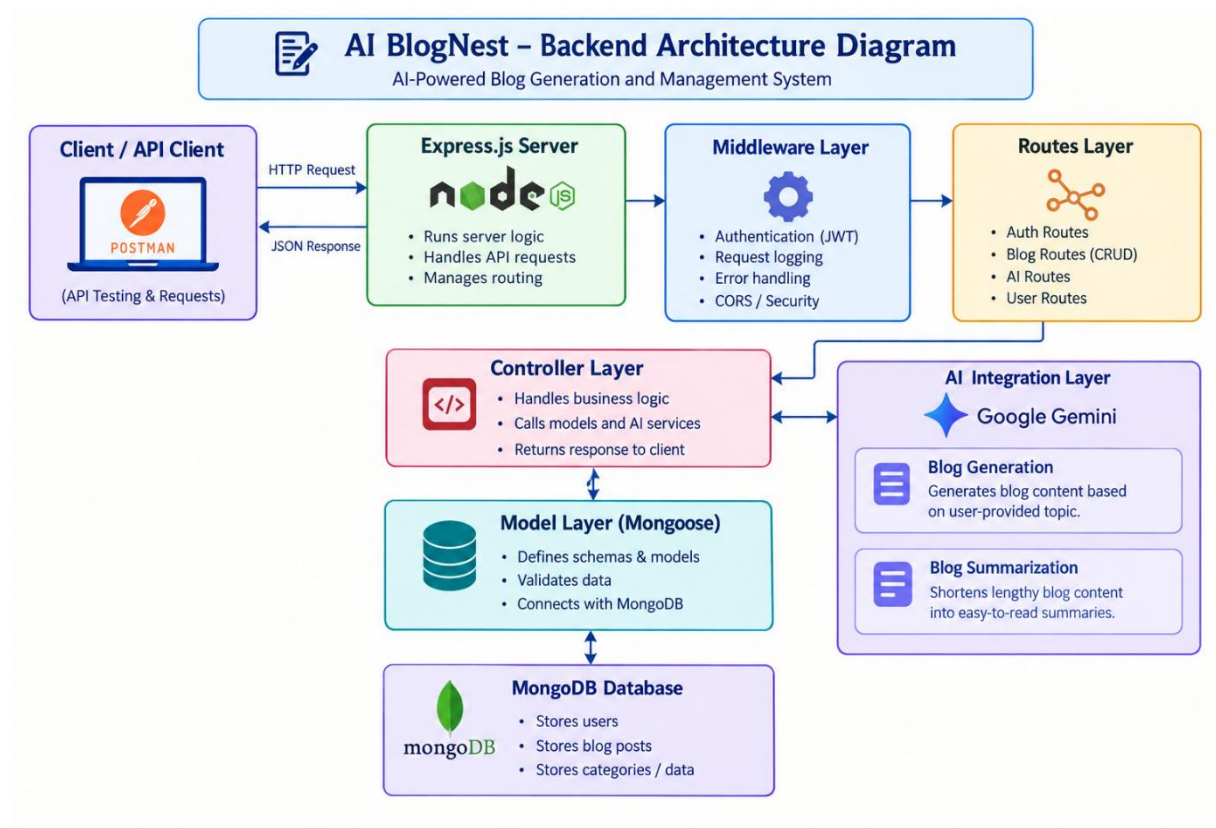
1. Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Linux – Provides the environment to develop and run the project.
- **Node.js (v16 or above):** Used to execute backend JavaScript code and run the server.
- **npm (v8 or above):** Used to install and manage project dependencies.
- **Express.js:** Used to create REST API endpoints and handle backend requests.
- **MongoDB:** NoSQL database used to store user details and blog information.
- **Mongoose:** Used to connect Node.js with MongoDB and manage database schemas.
- **Google Gemini AI:** Used to generate blog content and summarize existing blog content.
- **Postman:** Used to test API requests and verify API responses.
- **Visual Studio Code:** Used to write, edit, and manage project code.

2. Hardware Requirements:

- **Processor:** Intel Core i5 (8th Generation or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum, 16 GB recommended for smooth multitasking.
- **Storage:** Minimum 1 GB of available disk space for project files and dependencies.
- **Internet Connection:** Required for Google Gemini AI integration and package installation.
- **Keyboard and Mouse:** Used for coding and navigating development tools.
- **Monitor / Display:** Used to view code, terminal output, and API testing results.

Backend Architecture Diagram:



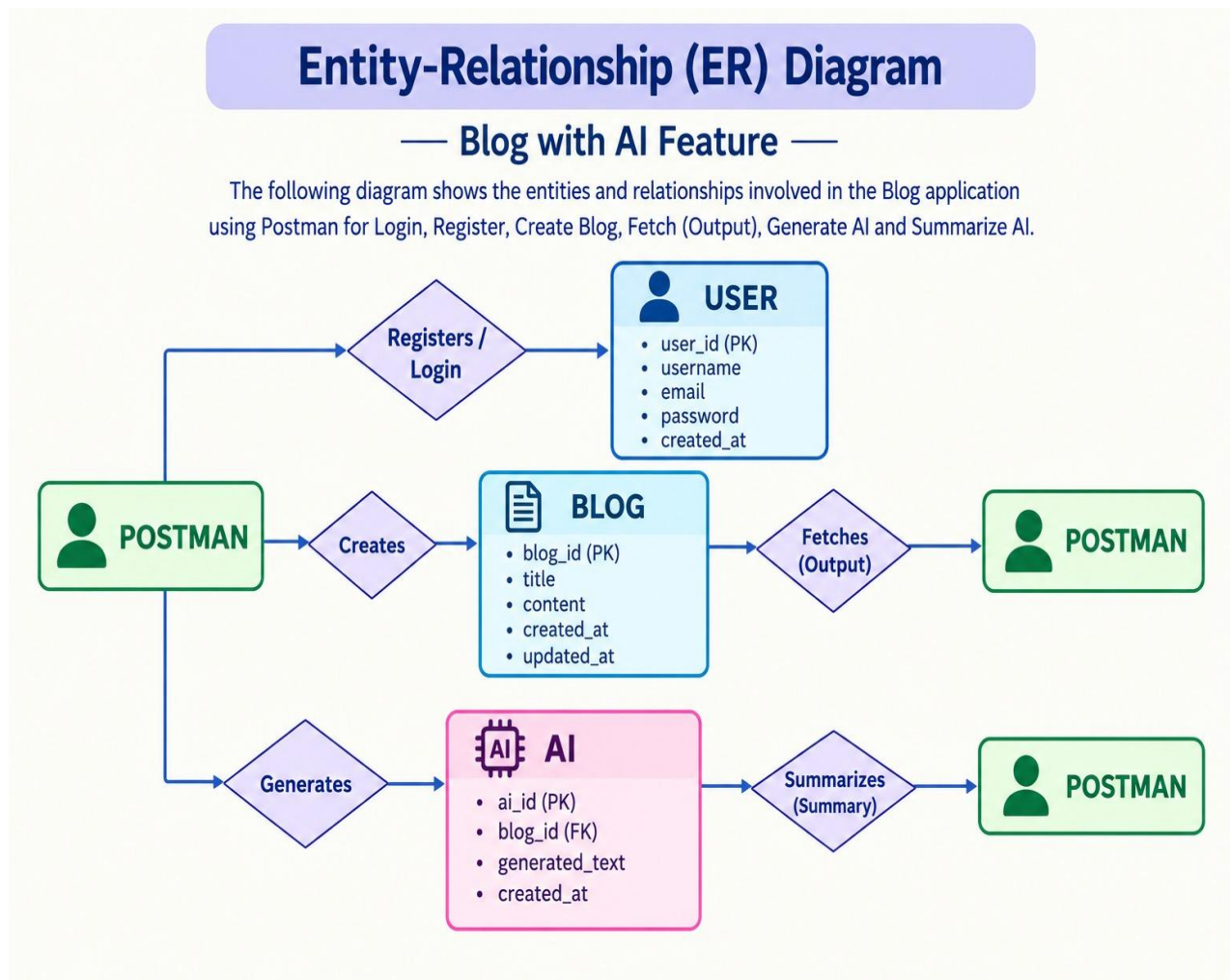
Backend Architecture Diagram – Explanation

- ✓ **API Client:** Sends requests to the backend and tests the APIs.
- ✓ **Express.js Server:** Receives and processes the client requests.
- ✓ **Middleware:** Provides authentication, security, and error handling.
- ✓ **Routes:** Directs each request to the required operation.
- ✓ **Controller:** Handles the main application logic and processes requests.
- ✓ **Mongoose Model:** Connects the application with the MongoDB database.
- ✓ **MongoDB:** Stores users and blog information.
- ✓ **AI Integration:** Uses Google Gemini AI to generate blogs and summarize blog content.
- ✓ **Response:** The processed result is sent back to the API client.

Components Definition:

1. **Interface:** The interface uses tools like Postman to send requests to the API. It helps test different routes and displays the response in JSON format.
2. **Express Server Gateway:** The Express server receives requests from the client and processes them. It handles different HTTP methods such as GET, POST, PUT, and DELETE.
3. **Authentication & Role Matrix Middleware:** The middleware checks the user's authentication details before allowing access. It also verifies the user's role and ensures that only authorized users can access protected resources.
4. **Granular Services / Logic Modules:** These modules contain the main rules and operations of the blog system. They manage different blog activities and control how each operation is processed.
5. **Database Interface Layer:** Mongoose acts as a connection between the application and Mongo DB. It helps the application store, retrieve, update, and manage the required data.
6. **AI Flow / Service Integration:** The AI Flow connects the application with external AI services. It supports features such as generating blog content and summarizing existing content.
7. **BlogGenerator with AI:** The BlogGenerator is used to create blog content automatically with the help of AI. It works with the AI Flow service to make the blog creation process easier and faster.

Entity-Relationship (ER) Diagram Description:



1. User Entity:

- **_name** (String, Required)
- **email** (String, Required, Unique)
- **password** (String, Hashed, Required)

2. Post Entity:

- **_id** (Primary Key - ObjectId)
- **userID** (Foreign Key referencing User Collection)
- **name** (String, Required - represents Title)
- **message** (String - Core content body)

Key Structural Mappings:

- ✓ **User to Post (One-to-Many):** An author can submit multiple blog documents.
- ✓ **User to Comment (One-to-Many):** Readers/Users can append multiple response comments across independent post nodes.
- ✓ **Post to Comment (One-to-Many):** A single blog document stores individual referenced comment submissions.

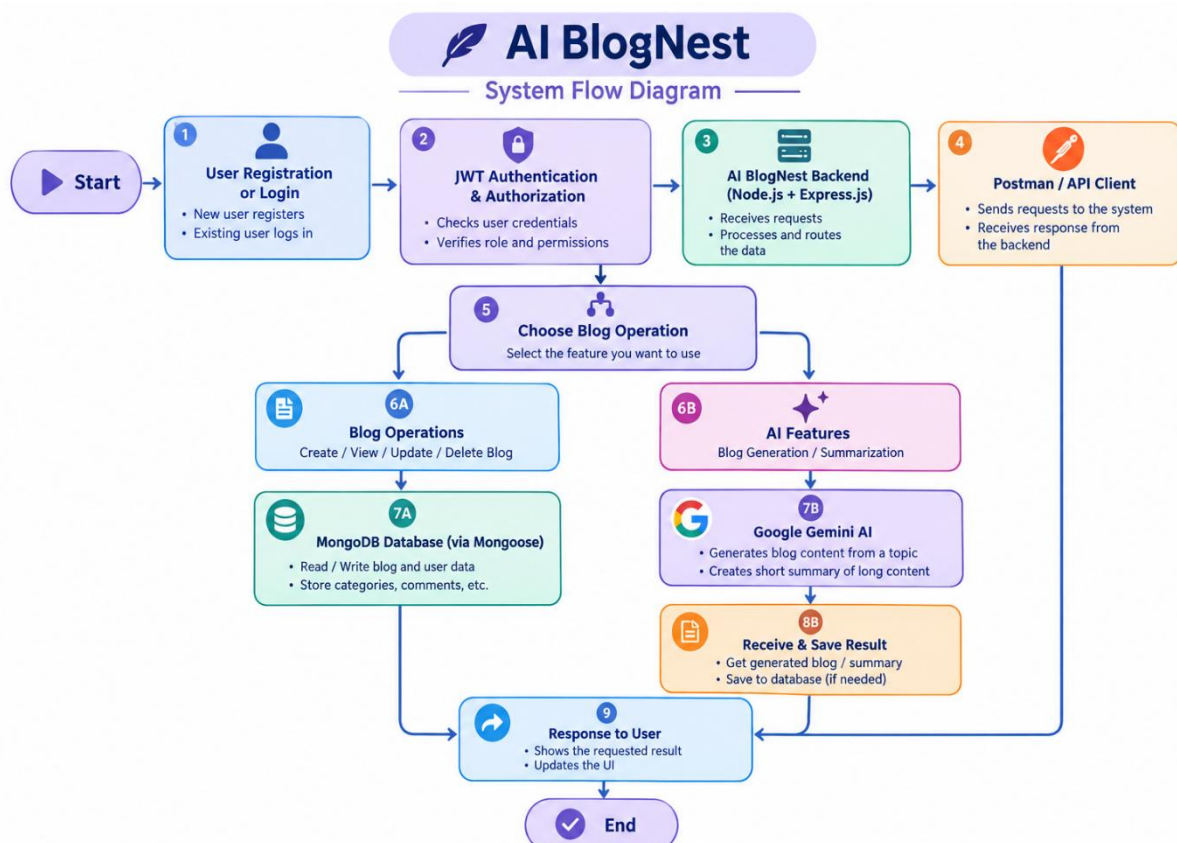
Key Features:

1. **Role-Based Access:** Controls user access based on their role and permissions. It helps protect restricted features from unauthorized users.
2. **Blog Management:** Allows users to create, view, update, and delete blog posts. It helps users easily manage their blog content.
3. **Category Management:** Organizes blogs into categories for easy searching and filtering. It makes related blog posts easier to find.
4. **Data Security:** Protects user and blog data through authentication and input validation. It helps keep the application and its data secure.
5. **AI Blog Generation:** Uses Google Gemini AI to generate blog content from a given topic. It reduces the effort required to create new blog posts.
6. **AI Content Summarization:** Uses AI to create short summaries of lengthy blog content.

ROLES AND RESPONSIBILITY:

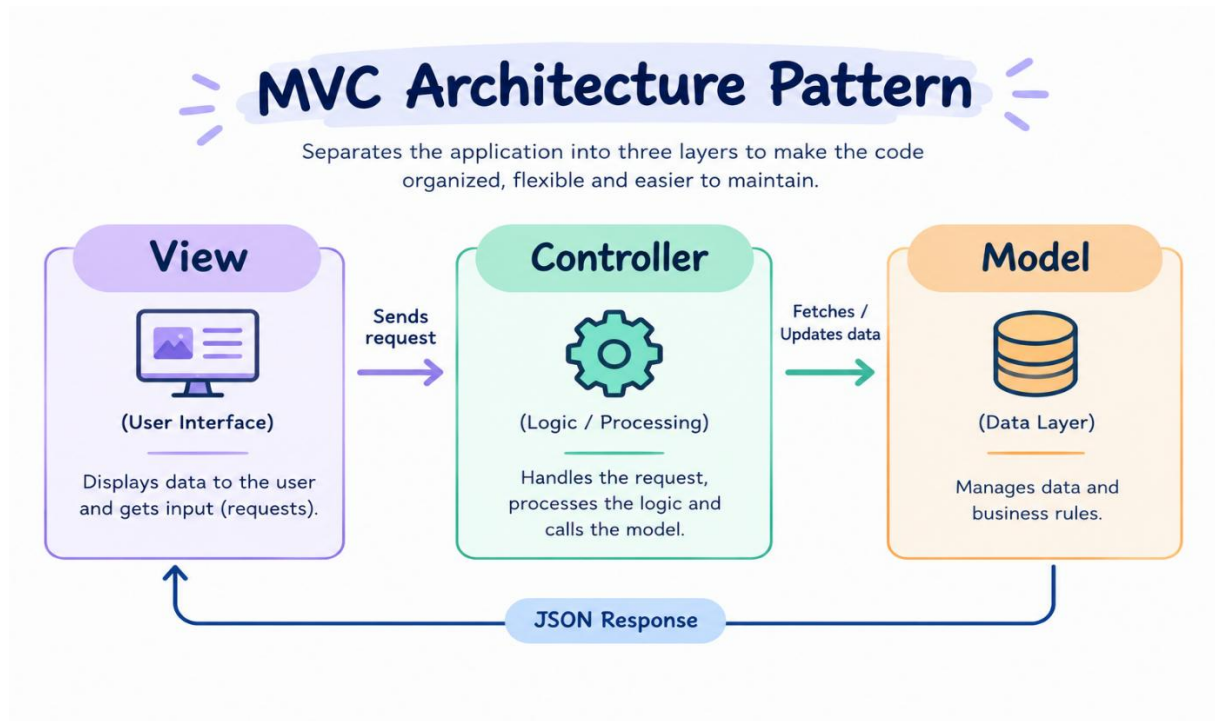
- **Admin:** Manages users, roles, blog posts, and comments in the system. The Admin has overall control over the application and its activities.
- **Editor:** Reviews and manages blog content and categories. The Editor helps maintain the quality and organization of blog posts.
- **Author:** Creates and manages their own blog posts. The Author can add new content and make changes to their posts.
- **Reader:** Reads published blog posts and can add comments. The Reader can interact with the available blog content.

FLOW DIAGRAM:



MVC Architecture Pattern:

- The system architecture follows the **Model-View-Controller (MVC) design pattern** to separate application responsibilities and maintain a modular, scalable, and maintainable backend structure.



Model Layer (Data Blueprint):

- The Model Layer defines the application's data structure using **Mongoose schemas**. These schemas specify the fields, data types, default values, validation rules, indexing requirements, and other data constraints for documents stored in **MongoDB**. The model layer is responsible for defining how application data is structured, validated, and persisted in the database.

Controller Layer (Business Logic):

- The Controller Layer acts as the **processing and coordination layer** of the application. It receives client requests from the API routes, validates the incoming request data, coordinates the required processing logic, interacts with the appropriate models, and prepares the resulting data. The controller then sends a **standardized JSON response** back to the client.

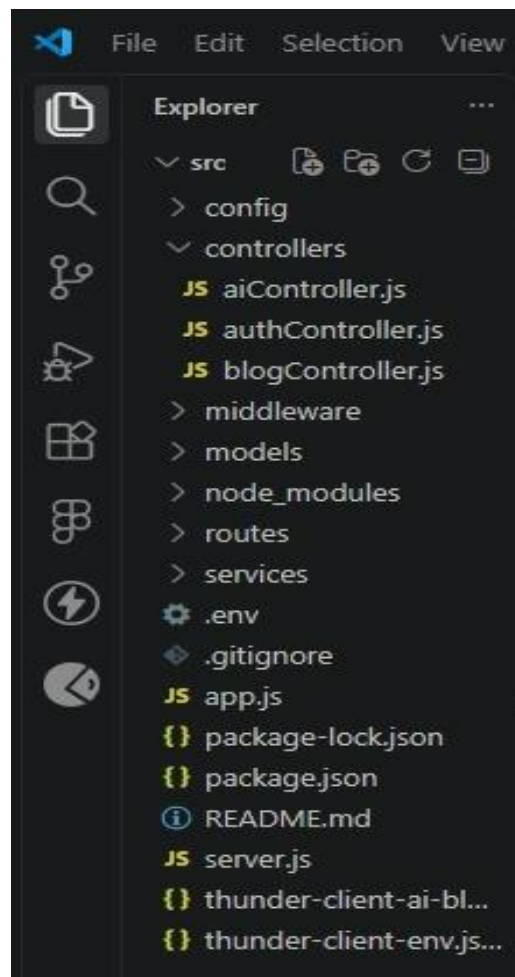
View Layer (API Routing Layer):

- Since this system is implemented as a **headless backend service**, it does not contain traditional user-interface templates or web pages. Instead, the interface is provided through an **API Routing Layer**. The routes define the available API endpoints and direct incoming HTTP requests to the appropriate controller handlers.

Project Setup:

➤ Creating project folder:

1. Create a new folder with your <project name>.
2. Inside that folder create one new folders.
3. Name another one as Server.
4. Now open that folder in VS Code.



Server setup (npm init)

- Open Server folder in terminal of VScode.
- `npm init -y`

Create files:

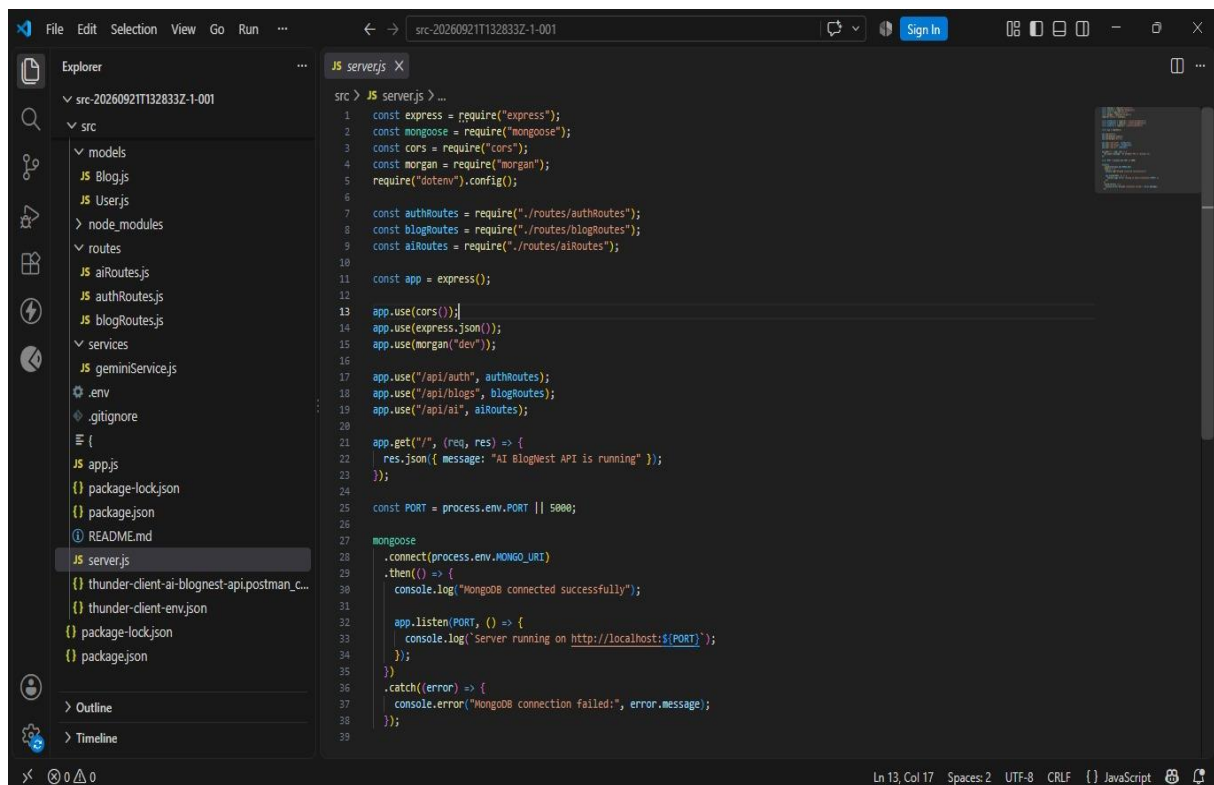
- `server.js`

Create folders:

- Models
- Controllers
- Routes

Backend Development:

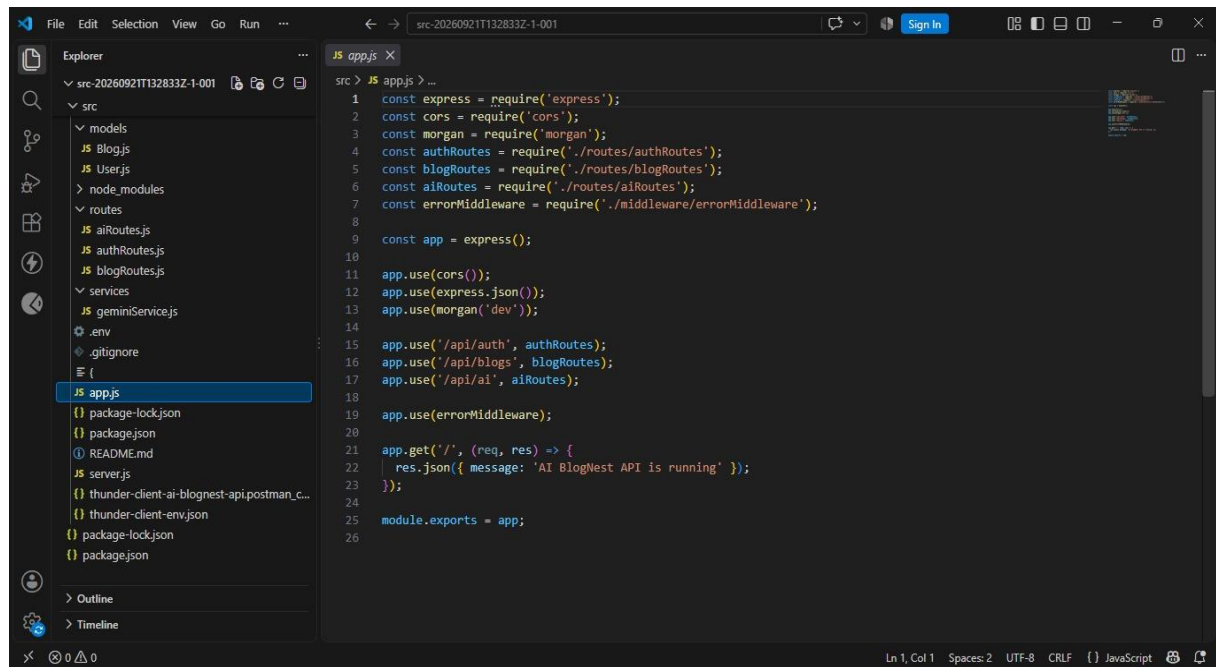
Server.js



The screenshot shows the VS Code editor interface. The Explorer sidebar on the left displays the project structure for 'src-20260921T132833Z-1-001'. The 'src' folder is expanded, showing subfolders 'models', 'routes', and 'services', along with files like 'app.js', 'package-lock.json', 'package.json', and 'server.js'. The 'server.js' file is selected and its content is displayed in the main editor area. The code in 'server.js' initializes an Express application, sets up middleware (cors, json, morgan), and routes for authentication, blogs, and AI. It also connects to a MongoDB database and listens on a specified port.

```
src > JS server.js > ...
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const cors = require("cors");
4  const morgan = require("morgan");
5  require("dotenv").config();
6
7  const authRoutes = require("../routes/authRoutes");
8  const blogRoutes = require("../routes/blogRoutes");
9  const aiRoutes = require("../routes/aiRoutes");
10
11  const app = express();
12
13  app.use(cors());
14  app.use(express.json());
15  app.use(morgan("dev"));
16
17  app.use("/api/auth", authRoutes);
18  app.use("/api/blogs", blogRoutes);
19  app.use("/api/ai", aiRoutes);
20
21  app.get("/", (req, res) => {
22    res.json({ message: "AI BlogNest API is running" });
23  });
24
25  const PORT = process.env.PORT || 5000;
26
27  mongoose
28    .connect(process.env.MONGO_URI)
29    .then(() => {
30      console.log("MongoDB connected successfully");
31
32      app.listen(PORT, () => {
33        console.log(`Server running on http://localhost:${PORT}`);
34      });
35    })
36    .catch((error) => {
37      console.error("MongoDB connection failed:", error.message);
38    });
39
```

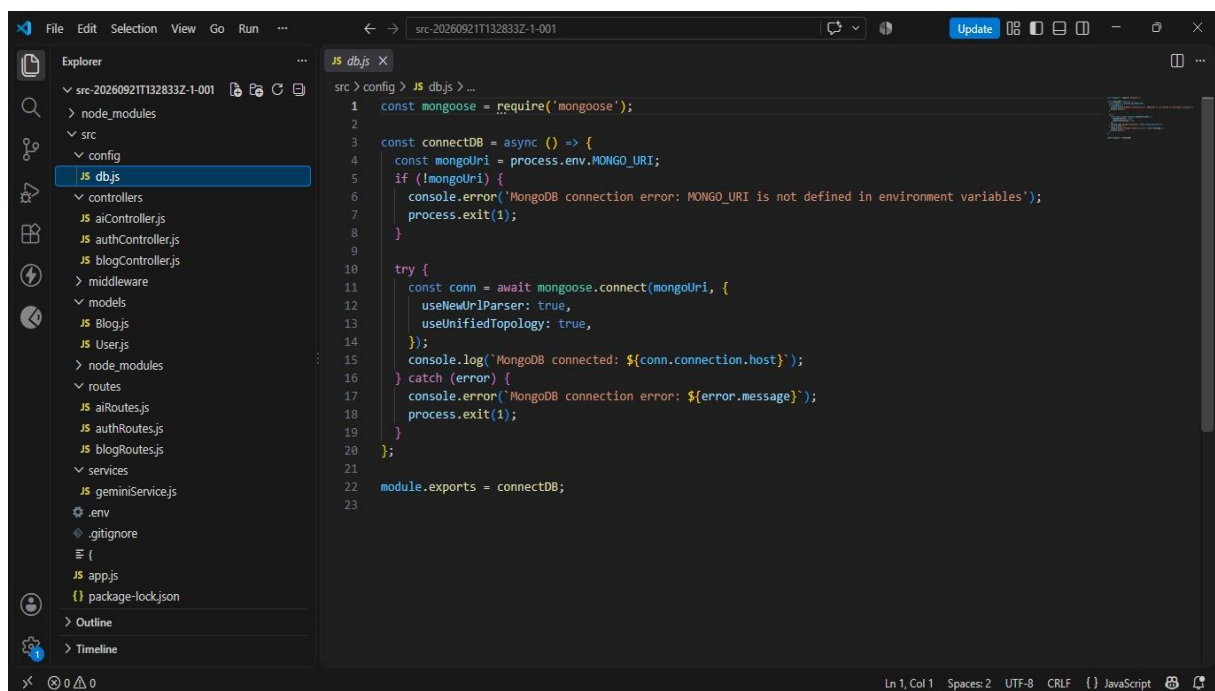
App.js



```
1 const express = require('express');
2 const cors = require('cors');
3 const morgan = require('morgan');
4 const authRoutes = require('./routes/authRoutes');
5 const blogRoutes = require('./routes/blogRoutes');
6 const aiRoutes = require('./routes/aiRoutes');
7 const errorMiddleware = require('./middleware/errorMiddleware');
8
9 const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
```

Database Configuration

DB Connection:-



```
1 const mongoose = require('mongoose');
2
3 const connectDB = async () => {
4   const mongoUri = process.env.MONGO_URI;
5   if (!mongoUri) {
6     console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7     process.exit(1);
8   }
9
10  try {
11    const conn = await mongoose.connect(mongoUri, {
12      useNewUrlParser: true,
13      useUnifiedTopology: true,
14    });
15    console.log(`MongoDB connected: ${conn.connection.host}`);
16  } catch (error) {
17    console.error('MongoDB connection error: ${error.message}');
18    process.exit(1);
19  }
20 };
21
22 module.exports = connectDB;
```

Configure MongoDB:

- ✓ Install Mongoose.

In your Node.js project directory, run:

`npm install mongoose`

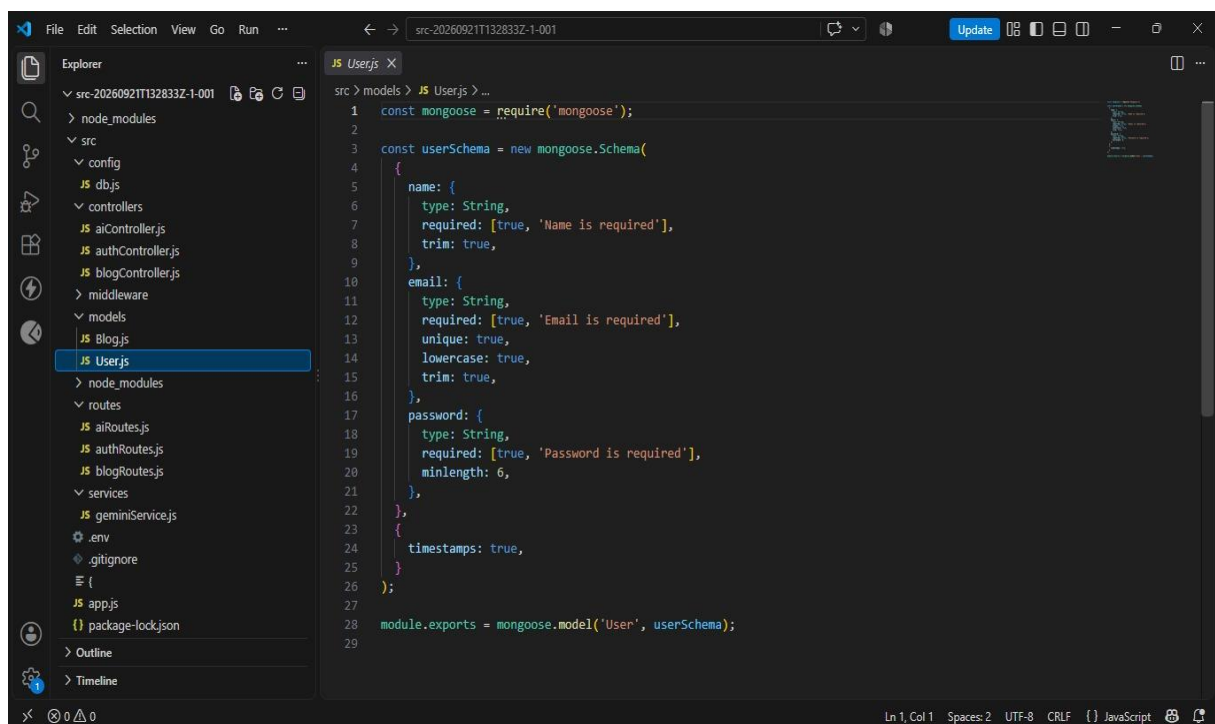
- ✓ Create database connection.
- ✓ Create Schemas & Models.

Create Database connection:

- Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions:

- Identity Resource Document Schema ([schemas/userModel.js](#))
- Enforces field criteria validations on incoming database creations:



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'src' directory containing 'models', 'controllers', 'routes', and 'services'. The 'models' directory is expanded, showing 'Blog.js' and 'User.js'. The 'User.js' file is selected and its content is displayed in the code editor. The code defines a Mongoose schema for a user model with fields for name, email, and password, each with specific validation rules. The schema is then used to create a model named 'User'.

```
1 const mongoose = require('mongoose');
2
3 const userSchema = new mongoose.Schema({
4   name: {
5     type: String,
6     required: [true, 'Name is required'],
7     trim: true,
8   },
9   email: {
10    type: String,
11    required: [true, 'Email is required'],
12    unique: true,
13    lowercase: true,
14    trim: true,
15  },
16  password: {
17    type: String,
18    required: [true, 'Password is required'],
19    minlength: 6,
20  },
21 }, {
22   timestamps: true,
23 });
24
25 module.exports = mongoose.model('User', userSchema);
```

Project Executions:

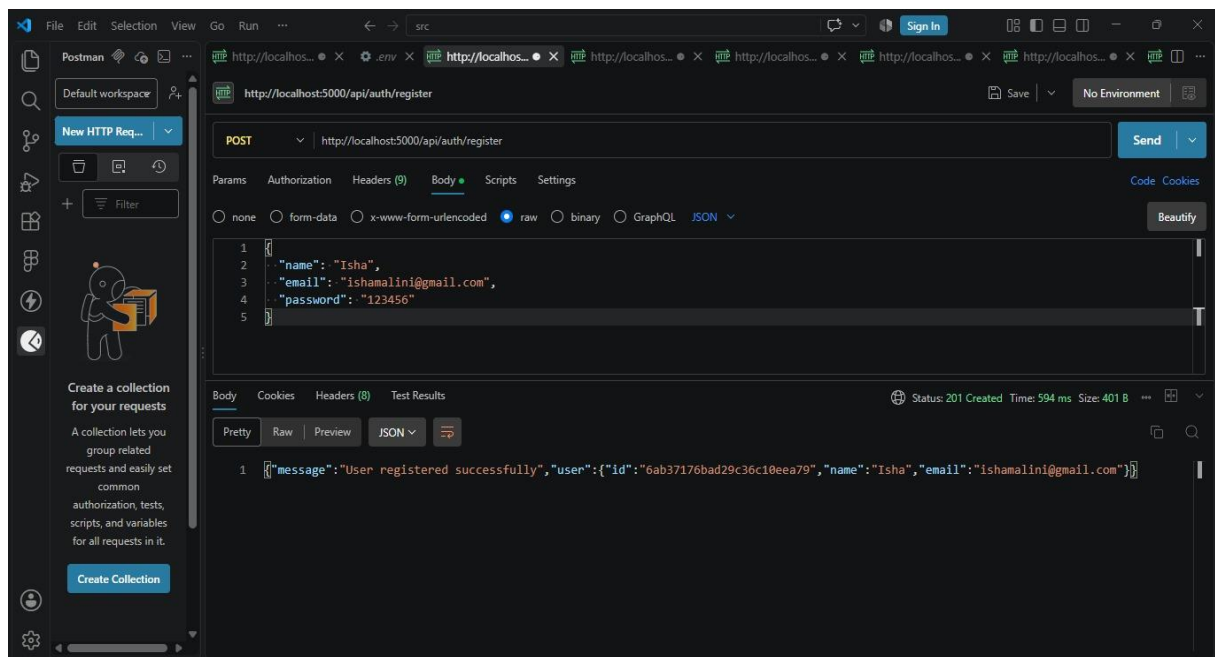
- Install Postman or ThunderClient (VS Code extension).

Start the backend server:

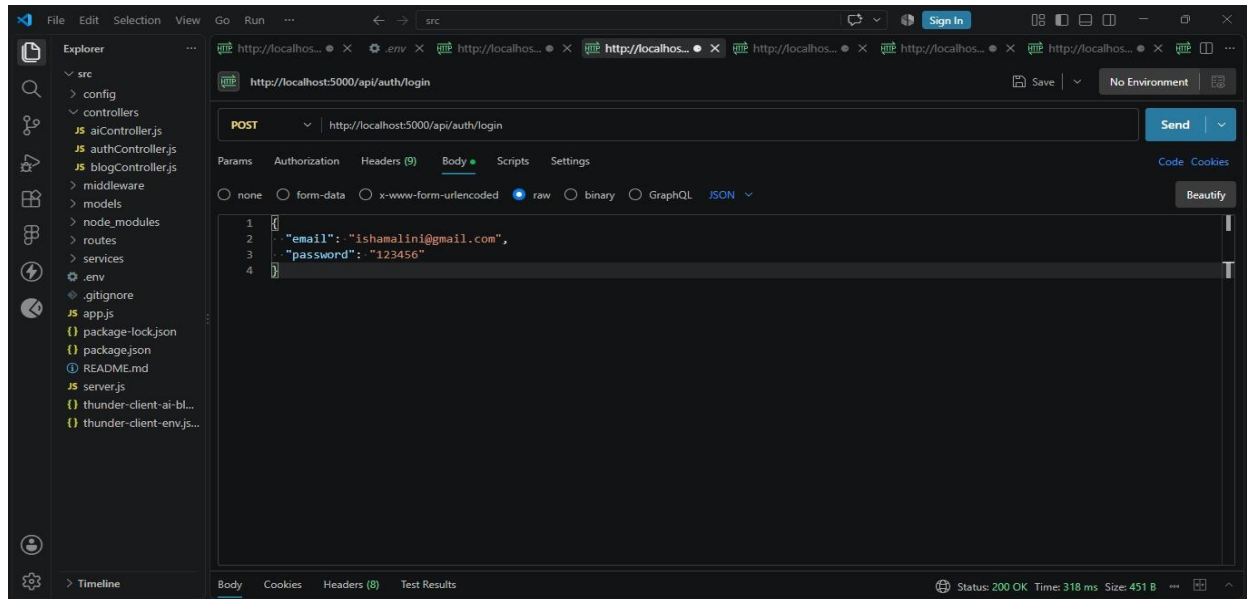
“npm start”

API Testing

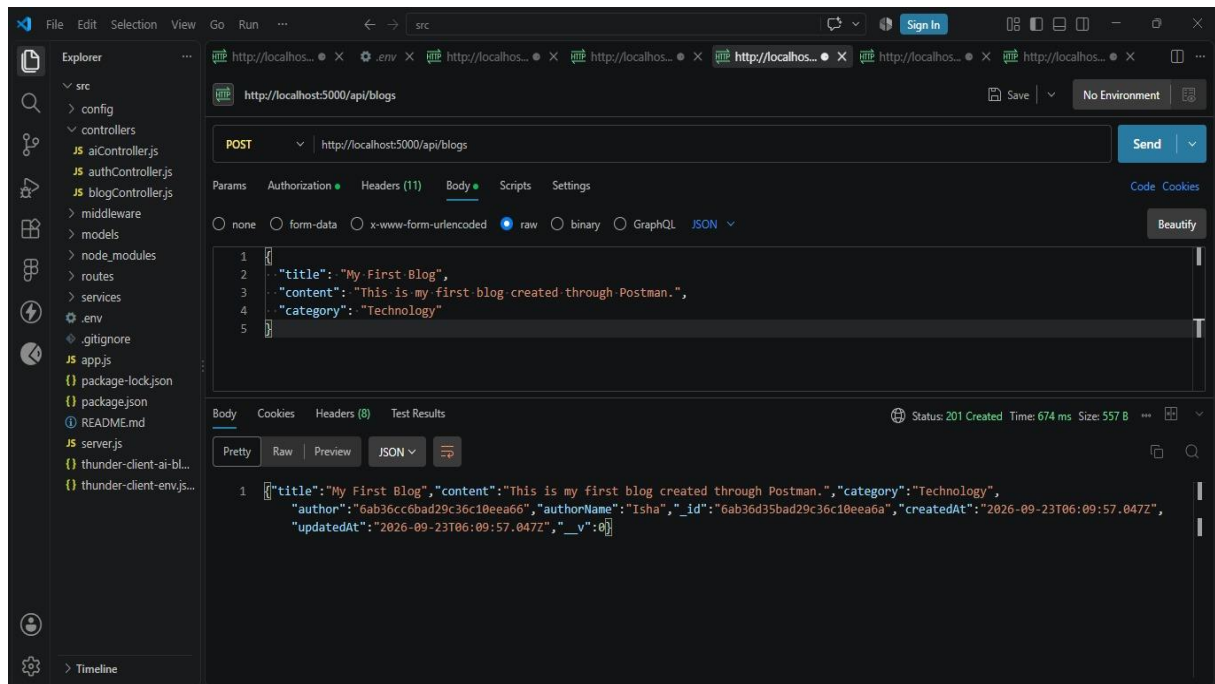
User Registration : POST: <http://localhost:5000/api/auth/register>



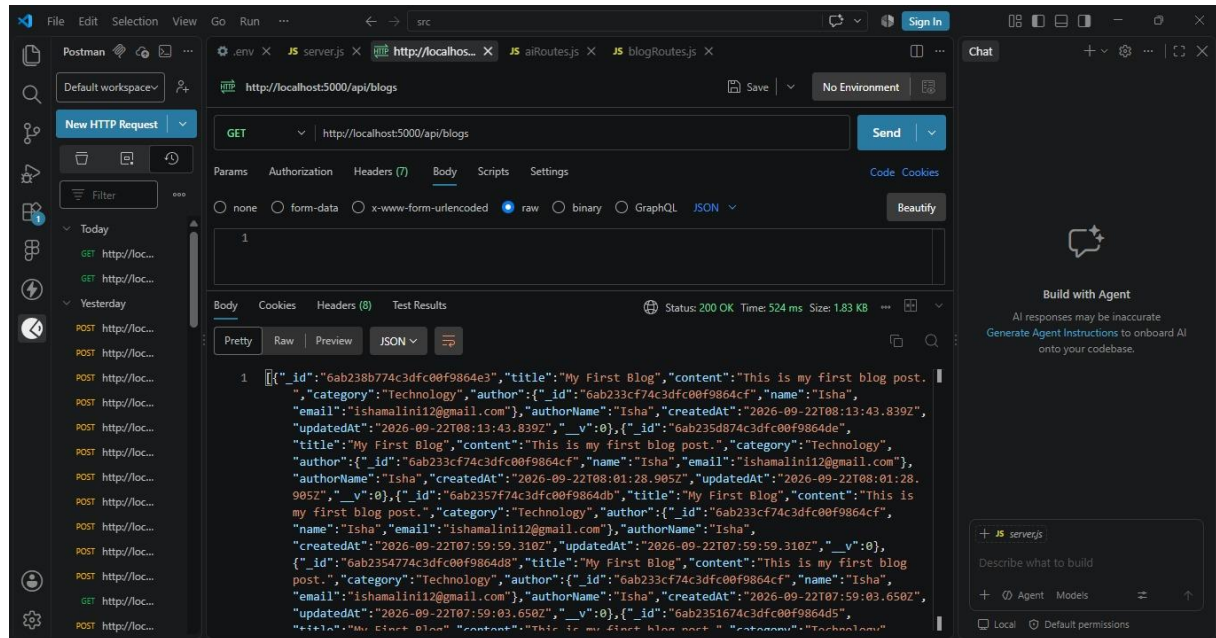
User Login: POST: http://localhost:5000/api/auth/login



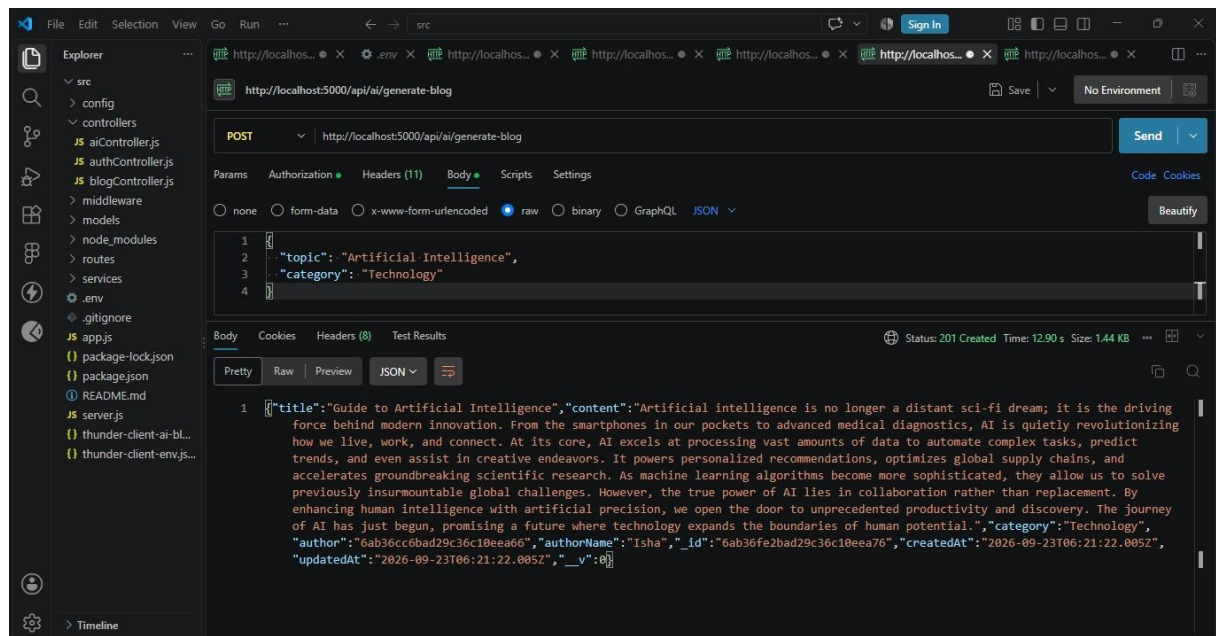
Create Blogs : POST: http://localhost:5000/api/blogs



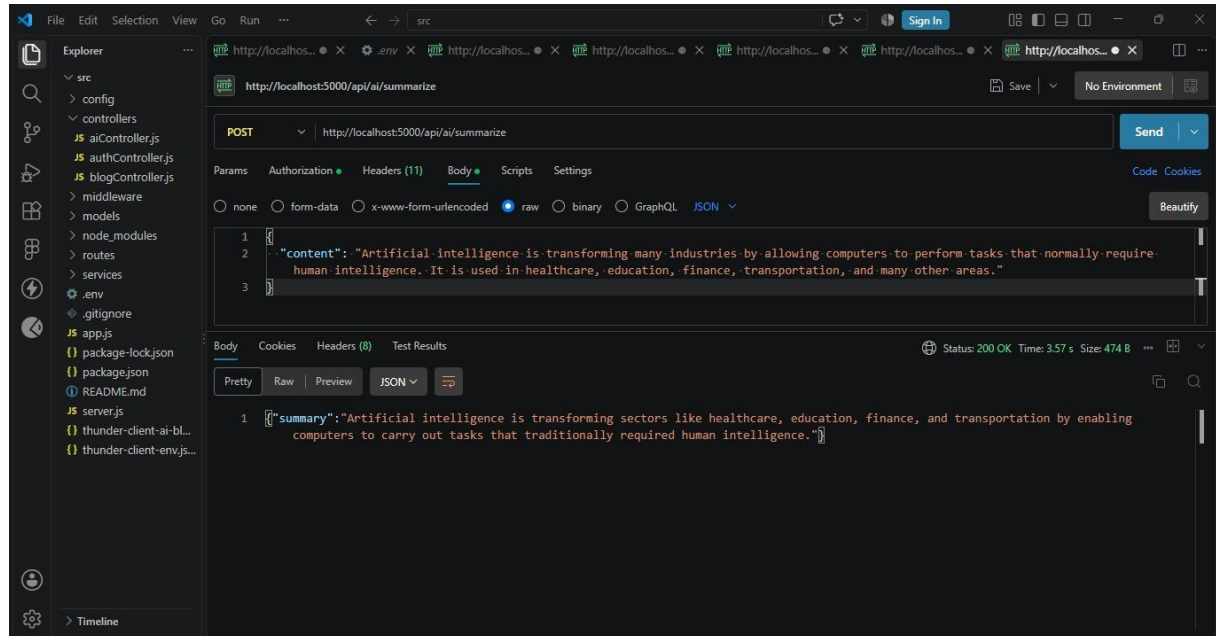
Fetch Blogs: GET: <http://localhost:5000/api/blogs>



Generate –AI Blogs: POST: <http://localhost:5000/api/ai/generate-blog>



Summary Blog : POST: <http://localhost:5000/api/ai/summarize>



DEMO VIDEO : [AI BLOG NEST API CODE VIDEO.mp4](#)

API TESTING: [AI BLOGNEST API.mp4](#)

